SQL

MBUA 512

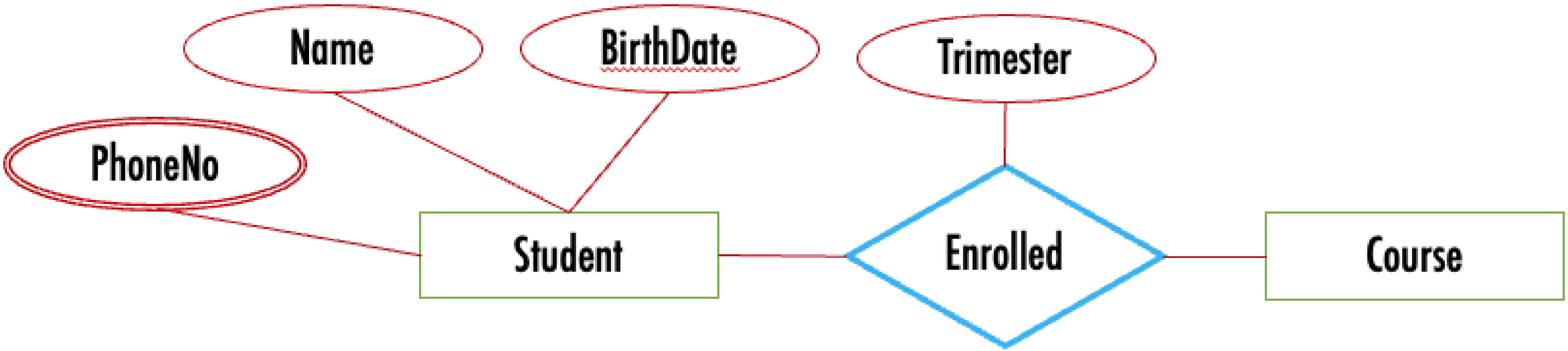
Class 3 – SQL

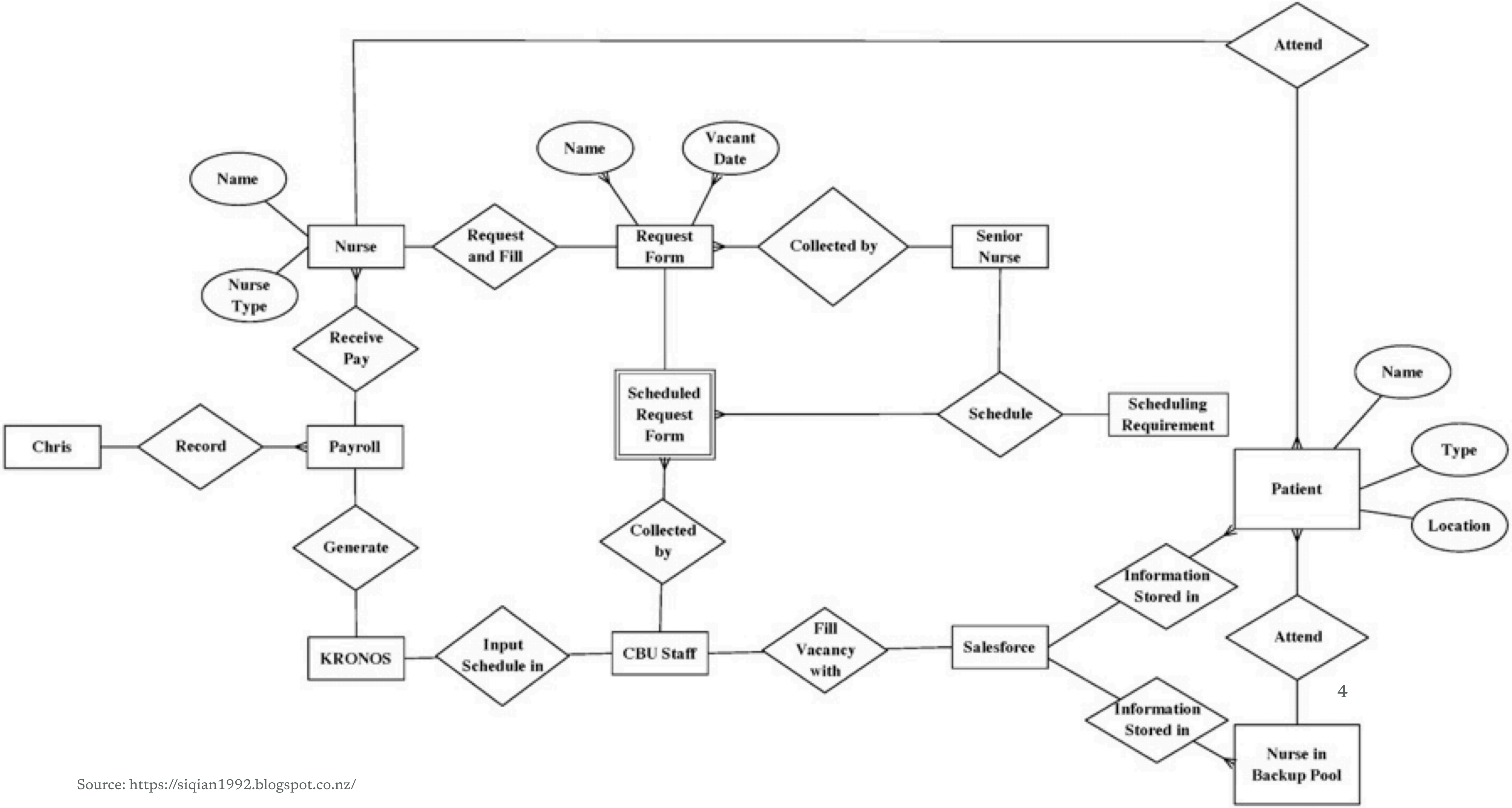
SQL = Structured Query Language, "es-queue-el" or "sequel". With Michael Winikoff.

Recap from last week (database design):

1. Data modelling using diagrams
2. Translating diagrams to relational databases, and the role of *keys*
3. Anomalies, and avoiding them by *normalisation*

Based on slides by Professor Markus Luczak-Roesch.


Name
BirthDate
Trimester
PhoneNo
Student
Enrolled
Course



Source: https://siqian1992.blogspot.co.nz/

SQL commands

- `CREATE TABLE` and `ALTER TABLE` - define the structure of a table (its *schema*)
- `INSERT INTO` and `DELETE FROM` and `UPDATE` - change the data in the table
- `SELECT` - query data

(there are a lot more than these)

> Important: make sure you select `MySQL` (not `Trino`) and your own workspace (`user_[username]`)

CREATE TABLE

```
CREATE TABLE name (
    column_name data_type,
    column_name data_type,
    ...
)
```

Data types

- Integer / Int – `12`, `-14`
- Varchar(length) – a string, `'Hello!'` (variable-length characters; use single quotes)
- Real – `3.14`, `-9.8345` (floating point, inexact)
- Decimal – `3.29` (exact, up to 38 digits)
- Varbinary – binary data
- Date – `DATE '2026-12-31'` (year-month-day, with the optional word `DATE`)
- Time – `TIME '01:58:36.77'` (with the optional word `TIME`; also a time-with-time-zone)

CREATE TABLE – example

```
CREATE TABLE people (
	personID INTEGER PRIMARY KEY AUTOINCREMENT,
	firstName VARCHAR(30) NOT NULL,
	lastName VARCHAR(30),
	birthdate DATE NOT NULL,
	planner INTEGER,
	email VARCHAR(30),
	FOREIGN KEY(planner) REFERENCES people(personID)
);
```

Constraints: PRIMARY KEY (and AUTOINCREMENT), FOREIGN KEY, NOT NULL

ALTER TABLE

`ALTER TABLE` changes the definition of an existing table.

```
ALTER TABLE name ADD COLUMN column_name data_type
ALTER TABLE people ADD COLUMN prefName VARCHAR

ALTER TABLE name DROP COLUMN column_name
ALTER TABLE people DROP COLUMN email -- loses data!

ALTER TABLE name RENAME COLUMN old_name to new_name
ALTER TABLE people RENAME COLUMN prefName to preferredName
```

INSERT INTO

`INSERT INTO table_name (columns...) VALUES (vals...), ...`

Single quotes are for strings, **double quotes** for database objects.

```
INSERT INTO people VALUES
         (1, 'Bob', 'Smith', '1917-01-03', 1, NULL),
         (2, 'Bob', 'Smith', '1973-03-25', NULL, 'bob@smith.net'),
         (3, 'Alice', 'Smith', '2026-07-07', 1, 'Alice@smith.net');
```

> the `(columns...)` is optional

Importing from a CSV

To generate rows from a CSV: ask an LLM, or use a spreadsheet with a few formulae (next slide).

> Prompt: "Convert the following CSV into an SQL statement to insert the data into an ANSI database"

Using a spreadsheet

- use `="'" & A2 & "'"` to convert each string cell into a quoted string
- use `="(" & TEXTJOIN(",",FALSE,D2:F2) & "), "` to join cells into a single row
- put `INSERT INTO people VALUES` in the first row
- copy and paste the rows
- demo ...

DELETE FROM table

```
DELETE FROM table WHERE conditions
```

> Leaving out the WHERE deletes **all** rows!

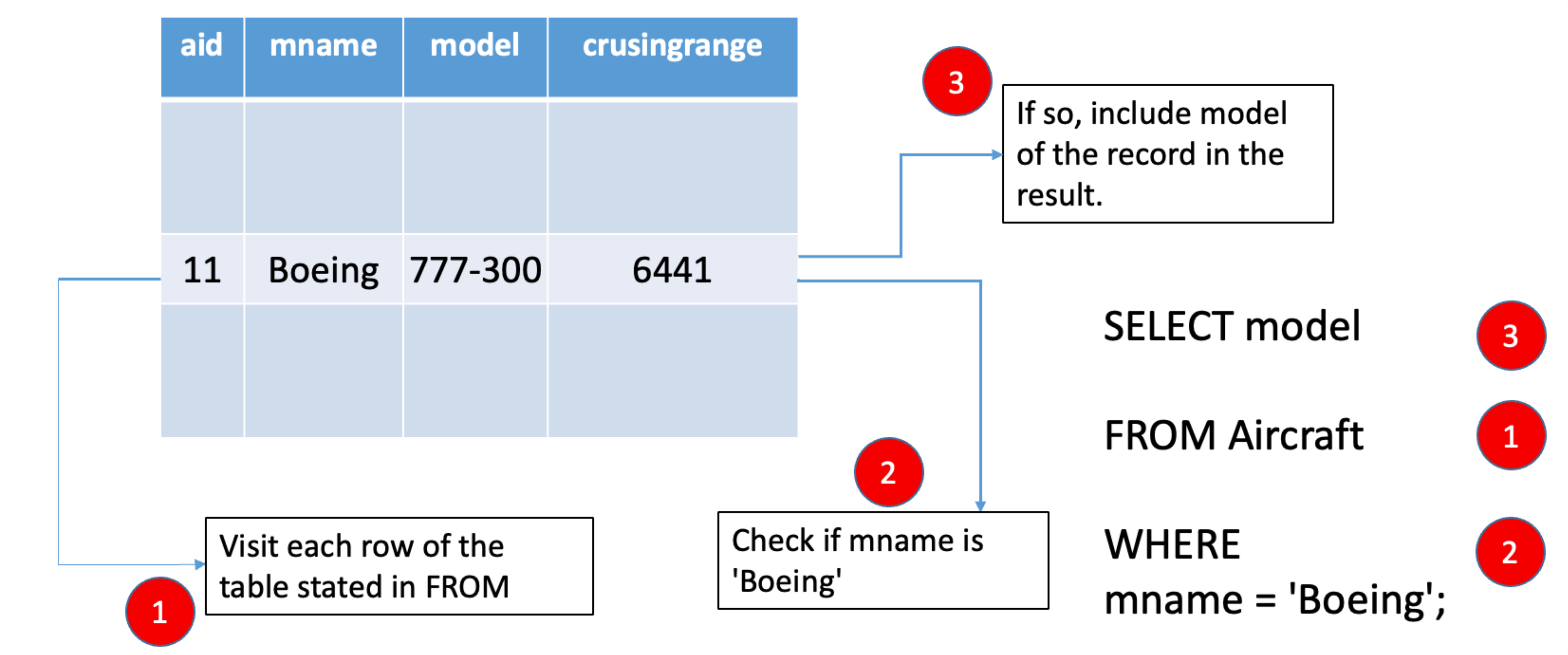
SELECT

```
SELECT what
FROM   table(s)
WHERE  conditions

-- "--" starts a comment to the end of the line
/* and this is a
  multi line comment */
SELECT *             -- "*" means "everything"
FROM   people
WHERE  email is NULL
```

By far the most complex SQL feature – covering basics today, more next week ...

How does SELECT work?



What about multiple tables?

Need to *join*

```
SELECT * FROM flight, pilot
```

Need a ***join condition***

A SQL query goes into a bar,
walks up to two tables and asks...

“Can I join you?”

Join condition

```
SELECT * FROM flight, pilot  WHERE flight.pilotID=pilot.pilotID
```

FLIGHT

Flight Number	Day	PilotID	Plane Model
NZ123	10 Aug 2018	PL8715	Airbus310
NZ456	11 Aug 2018	PL8716	Airbus320
YA789	17 Aug 2018	PL9781	Boeing737
NZ123	12 Aug 2018	PL9781	Boeing737
N124	13 Aug 2018	PL9781	Airbus310
YA890	17 Aug 2018	PL9781	Airbus310

PILOT

PilotID	PilotName
PL8715	Pat Smith
PL8716	Yi Zhang
PL9781	Jim Belich

Join condition - query result

```
SELECT * FROM flight, pilot  WHERE flight.pilotID=pilot.pilotID
```

Flight Number	Day	Pilot ID	Plane model	Pilot Name
NZ123	10 Aug 2018	PL8715	Airbus310	Pat Smith
NZ456	11 Aug 2018	PL8716	Airbus320	Yi Zhang
YA789	17 Aug 2018	PL9781	Boeing737	Jim Belich
NZ123	12 Aug 2018	PL9781	Boeing737	Jim Belich
NZ124	13 Aug 2018	PL9781	Airbus310	Jim Belich
YA890	17 Aug 2018	PL9781	Airbus310	Jim Belich
…	…	…	…	…

PILOT

PilotID	PilotName
PL8715	Pat Smith
PL8716	Yi Zhang
PL9781	Jim Belich

AIRCRAFT

Plane Model	Plane manufacturer
Airbus310	Airbus
Airbus320	Airbus
Boeing737	Boeing

FLIGHTINFO

Flight Number	Airline	Time
NZ123	Air NZ	10am
NZ456	Air NZ	11am
YA789	Yugoslav Air	10:30am
NA124	Air NZ	11pm
YA890	Yugoslav Air	4pm

FLIGHT

Flight Number	Day	PilotID	Plane Model
NZ123	10 Aug 2018	PL8715	Airbus310
NZ456	11 Aug 2018	PL8716	Airbus320
YA789	17 Aug 2018	PL9781	Boeing737
NZ123	12 Aug 2018	PL9781	Boeing737
N124	13 Aug 2018	PL9781	Airbus310
YA890	17 Aug 2018	PL9781	Airbus310

Summary

- `CREATE TABLE` and `ALTER TABLE` – define the structure of a table (its *schema*)
- `INSERT INTO`, `DELETE FROM`, `UPDATE` – change the data in the table
- `SELECT` – query data

Next: activities in part 2 today – learn by doing. Next week, more SQL, focusing on `SELECT`.

Activities

Activities – SQL (1 of 3)

1. Go to learn.datascie.nz and login if needed
2. Select `MySQL` (not `Trino`) and your own workspace (`user_[username]`)

Activities – SQL (2 of 3)

Practise using SQL to create a database, add content, and explore constraints

3. CREATE a DB for the student-advisor database (see slide 22 of last week) - you will need to select appropriate data types
4. Add content the your DB using INSERT INTO - you will need to make up data (save the query so you can re-insert the data after deleting it)
5. Delete the content using DELETE
6. Create a CSV file (using a spreadsheet, e.g. Excel or Google Sheets) and insert it into your database (note: each sheet corresponds to a single table)
7. Explore constraints: what happens if you try and insert a duplicate row or use a non-existent foreign key?

Activities – SQL (3 of 3)

Practise writing SQL queries using SELECT

8. Create the flight DB using this file
9. Write a simple SELECT query to find all flight numbers flown by AirNZ (hint: use the flightinfo table)
10. Write a SELECT query to find all pilotIDs who fly for AirNZ (hint: combine the flight and flightinfo tables, building on your previous query)
11. Modify the previous query to find the *names* of all pilots who fly for AirNZ (hint: join three tables - flight, flightinfo, and pilot)